do the same thing. However, we also cared about visualizing the attention weights to see what exactly the agent has learned to focus on when playing the game. Some of the results are surprising, and some are what we would expect.

To visualize the attention weights, we had our model save a copy of the attention weights each time it was run forward, and we can access it by calling `GWagent.att_map`, which returns a *batch* × *head* × *height* × *width* tensor. All we need to do is run the model forward on some state, select an attention head, and select a node to visualize, and then reshape the tensor into a 7 × 7 grid and plot it using `plt.imshow`.

```
>>> state_ = env.reset()
>>> state = prepare_state(state_)
>>> GWagent(state)
>>> plt.imshow(env.render('rgb_array'))
>>> plt.imshow(state[0].permute(1,2,0).detach().numpy())
>>> head, node = 2, 26
>>> plt.imshow(GWagent.att_map[0][head][node].view(7,7))
```

We decided to look at the attention weights for the key node, the door node, and the agent node to see which objects are related to each other. We found the node in the attention weights that corresponds to the node in the grid by counting the grid cells, since both the attention weights and the original state are a 7 × 7 grid. We intentionally designed the relational module such that the original state and attention weight matrices are the same dimensionality; otherwise it becomes difficult to map the attention weights onto the state. Figure 10.23 shows the original full view of the grid in a random initial state and the corresponding prepared state.

Figure 10.23. Left: The full observation of the environment. Right: The corresponding partial state view that the agent has access to.